

AGENTIC OS V2

Diagrama de despliegue y procesamiento concurrente



Separación de responsabilidades. Las ventanas envían las letras directamente a IALearner. El launcher no transporta el texto: administra procesos locales y conserva un canal de control para coordinar la sesión y recibir el tipo de usuario inferido.

Arquitectura y despliegue

La solución utiliza una arquitectura cliente-servidor. **ialearner** representa el data center; **launcher** y los procesos **window** representan el sistema del usuario. El launcher crea cada ventana mediante *fork* y *exec*, consulta su estado con *waitpid* y termina únicamente sus procesos locales. Cada ventana captura eventos de teclado con X11 y transmite cada carácter por un socket TCP. Al recibir *return*, el hilo remoto asociado a esa ventana termina la oración y la inserta en la cola de su sesión.

IALearner escucha el canal de control en el puerto 5001 y asigna servicios de ventana desde el puerto 5100. Cada launcher posee un *SessionContext* independiente, con sus propias ventanas, cola de oraciones, pool de detectores y perfil; por ello los datos de sesiones distintas no se mezclan. Para la demostración, los componentes se ejecutan en dos terminales de la misma computadora usando 127.0.0.1.

Canales de comunicación

Origen	Destino	Información	Finalidad
Launcher	Window local	Creación y señales	Administración de procesos
Window	IALearner	Letras y return	Formación remota de oraciones

Procesamiento concurrente



Loader y pool permanente

El parámetro **P** se recibe al iniciar IALearner y se limita al número de CPU disponibles. Para cada sesión se crean una sola vez **P hilos detectores**. Estos workers permanecen suspendidos sobre una variable de condición mientras no tengan tareas. El Loader también duerme, sin consumir CPU, hasta que la cola acumula P oraciones; entonces extrae el lote, publica una oración para cada worker y despierta al pool. Los detectores ejecutan el algoritmo simultáneamente y, al terminar, vuelven a quedar suspendidos.

Si llegan más de P oraciones, el excedente permanece en la cola mientras se procesa el lote activo. Al completarse, el Loader reactiva el pool con el siguiente grupo de hasta P tareas. Durante el cierre se procesa también el remanente inferior a P. Los mutex separan las secciones críticas de cola, lote, perfil, sesión e impresión; las variables de condición evitan espera activa.

Clasificación y decisión asíncrona

Cada oración se normaliza a minúsculas y se tokeniza mediante **strtok_r**, versión reentrante necesaria porque varios detectores clasifican simultáneamente. La técnica *bag of words* cuenta coincidencias con los diccionarios de correo electrónico, artículo científico y reporte. Una clase es candidata al reunir al menos tres palabras; si hay varias candidatas, se elige la de mayor frecuencia.

Apenas un worker determina la clase, actualiza el perfil protegido por mutex y evalúa la tabla de decisión, sin esperar que termine el lote completo. De esta manera la inferencia del usuario es concurrente y asíncrona respecto de las demás oraciones.

Documentos detectados	Tipo de usuario
Correo electrónico	Personal administrativo
Correo electrónico + Reporte	Personal técnico
Correo electrónico + Artículo científico	Profesor
Artículo científico + Reporte	Estudiante

Robustez y liberación de recursos

La implementación valida parámetros, memoria, sockets e hilos. Utiliza TDAs apropiadas: tabla hash para diccionarios, cola circular para oraciones y gestores para procesos e hilos. Al finalizar se despiertan y unen el Loader y los detectores, se cierran sockets, se esperan procesos y se liberan estructuras dinámicas, evitando hilos suspendidos, procesos zombis y condiciones de carrera.